

**TOWARDS SECURING SYSTEMS: LEVERAGING GENERATIVE AI FOR ENHANCED
VULNERABILITY DETECTION AND CORRECTION**

A THESIS

Presented to the Department of Computer Engineering and Computer Science
California State University, Long Beach

In Partial Fulfillment
of the Requirements for the Degree
Master of Science in Computer Science

Committee Members:

Muhammad Abdul Basit Ur Rahim, Ph.D. (Chair)
Mehrdad Aliasgari, Ph.D.
Tairan Liu, Ph.D.

College Designee:

Kerop Janoyan, Ph.D.

By ILMAAN ZIA

Jamia Hamdard University

October 2024

Copyright © 2024

Ilmaan Zia

All Rights Reserved.

ABSTRACT

In an era of rapidly evolving cyber threats, the need for robust, efficient, and intelligent vulnerability detection tools has become paramount. This thesis, titled "TOWARDS SECURING SYSTEMS: ENHANCING VULNERABILITY DETECTION AND BUILDING AI POWERED DETECTION TOOLS" addresses the critical gap in cybersecurity infrastructure, particularly in the realm of automated vulnerability detection and mitigation. As software systems grow in complexity, especially in emerging fields like robotics, traditional security measures often fall short. This research aims to develop advanced, AI-driven solutions to identify and resolve security vulnerabilities more effectively, responding to the urgent demand for proactive security measures in an increasingly interconnected digital landscape.

To achieve this goal, our study began with the development of a comprehensive vulnerability dataset. Recognizing the limitations of existing small-scale datasets, we employed web scraping techniques and generated synthetic data, with a particular focus on robotic systems. This approach significantly expanded the scope and diversity of the dataset, encompassing a wide range of vulnerability types that can potentially compromise system security.

Building upon this enhanced dataset, we then focused on fine-tuning large language models using carefully crafted, vulnerability-specific prompts. This process aimed to create a more specialized and accurate model for vulnerability detection and analysis.

The aim of this research is the development of an innovative application that leverages the fine-tuned language model. This application consists of two primary modules:

An interactive chatbot interface that allows users to input code directly for real-time vulnerability assessment. The system provides comprehensive feedback, including vulnerability status, Common Weakness Enumeration (CWE) identification, detailed reasoning, and suggestions for secure code alternatives.

A file scanning module that enables users to upload Python files for automated vulnerability analysis. This module generates detailed PDF reports summarizing file contents and associated

vulnerabilities, offering a scalable solution for larger codebases.

This thesis contributes to the field of cybersecurity by presenting a novel approach to vulnerability detection and mitigation, combining advanced machine learning techniques with practical, user-friendly tools. The research demonstrates the potential of AI-powered systems in enhancing software security, particularly in the context of increasingly complex and vulnerable robotic systems.

ACKNOWLEDGEMENTS

I want to sincerely thank my thesis advisor, Dr. Abdul Basit Ur Rahim, for all of his help and guidance during the study process. Dr. Basit was a major influence on how I approached my research, and his perseverance and commitment were crucial in helping me finish my thesis.

I also express my heartfelt gratitude to Dr. Merhdad Aliasgari and Dr. Tairan Liu for their invaluable input and for graciously serving on my thesis defense committee. Their expertise and constructive feedback have significantly contributed to the quality of this work.

To my dear friend, especially Ankit Raj, whose collaborative assistance, understanding, encouragement, has played a significant role in my ability to persevere and succeed.

Finally, my wonderful parents have my lifelong gratitude. You have taught me to be persistent and hardworking, and that our bravery and tenacity determine our fate. My current self is a direct result of your advice and encouragement, and I want to make you proud.

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGEMENTS	iii
LIST OF TABLES	v
LIST OF FIGURES	vi
1. INTRODUCTION	1
2. BACKGROUND	7
3. DATASET ENHANCEMENT & PREPARATION FOR VULNERABILITY DETECTION	15
4. FROM GENERAL TO SPECIALIZED: FINE-TUNING LLMS FOR ROBUST VULNERABILITY ANALYSIS	22
5. VULSCANNER - SPECIALIZED VULNERABILITY DETECTION APPLICATION	34
6. CONCLUSIONS AND FUTURE WORKS	39
APPENDIX:	42
REFERENCES	46

LIST OF TABLES

1.	Common Code Vulnerabilities in Robotic Systems	8
2.	Overview of Dataset Vulnerable and Non-Vulnerable Data	16
3.	Pre Fine-tune Evaluation on Insecure Examples	25
4.	Comparison of Evaluation Metrics	32

LIST OF FIGURES

1.	What does fine-tuning do for the model	10
2.	Dataset Statistics Post Processing	21
3.	Prompts Designed for Desired Input And Output	26
4.	Post Fine-tuning Results on Vulnerable Data	28
5.	Alphacodium Flow	29
6.	Pipeline for Vulnerability Detection	31
7.	Top 6 CWEs Comparative Analysis	33
8.	Chatbot User Interface	36
9.	File Scanner User Interface	37
10.	Workflow of Chat Module	43
11.	Generated PDF Report For a Python File	45

CHAPTER 1

INTRODUCTION

The only system which is truly secure is one which is switched off and unplugged, locked in a titanium lined safe, buried in a concrete bunker, and is surrounded by nerve gas and very highly paid armed guards. Even then, I wouldn't stake my life on it.

— Gene Spafford, Professor of Computer Science at Purdue University

Robust, effective, and intelligent vulnerability detection systems have become increasingly important in today's fast changing digital ecosystem. Identifying and addressing potential vulnerabilities in software systems is a challenge for standard security methods, particularly in developing sectors like robotics where complexity is increasing. [1] [2]. Today, software development is integral to nearly all technological advancements, making the quality and security of source code increasingly critical. As evidence of this growing reliance, the number of Github repositories surged from 100 million in 2018 to 200 million in 2022 [3]. Early detection and elimination of vulnerabilities is crucial for organizations to protect them from possible threats and malicious attacks. Early detection and elimination of vulnerabilities is crucial for organizations to protect them from possible threats and malicious attacks.

Studies from organizations like MITRE¹ and other research indicate that there has been a significant rise in vulnerabilities in source code over recent years. Code vulnerabilities or software vulnerabilities are particularly concerning as they represent weaknesses that can be exploited to breach security policies, leading to severe consequences such as data leakage, data tampering, or denial of service attacks [4–9].

Static source code analysis is a technique for identifying vulnerabilities within code by automatically inspecting the source code without executing the program. This approach, known as

¹<https://cve.mitre.org/>

Static Application Security Testing (SAST), involves analyzing either the source code or a compiled version to uncover security issues. Two critical features of SAST tools are their ability to cover a wide range of errors and achieve high accuracy. Prominent SAST tools like Semgrep, Bandit, and SonarQube typically rely on rule-based methods to detect vulnerable code patterns. [10]

In recent years, Machine Learning and deep learning techniques have achieved significant advancements in natural language processing [11] [12]. Deep learning models are well suited for code understanding and processing tasks given the structural similarities between code and natural languages. Consequently, researchers have been growing interest in applying deep learning techniques to vulnerability detection [13], [14].

ML models can autonomously learn and identify patterns of software vulnerabilities from extensive datasets. Studies have demonstrated that these models tend to produce fewer false positives compared to traditional Static Application Security Testing tools [6], [7]. Recent research highlights the superior performance of deep learning models over several open-source tools for C/C++ code, showing a reduction in both false positive and false negative rates [2].

However, identifying vulnerabilities at a finer level, such as targeting vulnerable statements and defining a reason, remains a challenge. Existing vulnerability detection tools [13] typically classify functions rather than identifying the reason for them. A recent study [14] has demonstrated that these tools tend to learn high-level vulnerable features and fail to highlight the reason for vulnerable statements and yet fail to provide a secure version of the code. To address these challenges, we developed a fine-tuned Large Language Model (LLM) specifically designed for vulnerability detection and correction in source code. By leveraging the capabilities of LangGraph, we created a multi-agent workflow that iteratively improves vulnerability detection and generates secure versions of code. Our approach uses a custom dataset for training, allowing the model to accurately classify code as vulnerable or non-vulnerable and identify specific vulnerable statements. Additionally, the model provides corresponding Common Weakness Enumeration (CWE) IDs and secure code suggestions, significantly reducing the need for manual code inspection and improving overall

detection accuracy.

Moreover, we worked considerably on expanding our dataset with web scraping and synthetic data generation to enhance the effectiveness of our model. In this way, by expanding, we were able to include other types of vulnerabilities, such as the ones connected to robotic systems that are becoming increasingly popular in several modern applications. The augmented dataset enhanced the training process of the model, but also allowed it to recognize and address a greater number of security issues, making it even more robust and reliable.

In parallel with the dataset enhancement, we also focused on the application aspect of our work. We developed a user-friendly application that integrates our fine-tuned LLM, providing developers with practical tools for vulnerability detection. The application consists of two primary modules: a file scanning module that lets users upload full Python files for in-depth analysis and a chatbot interface that lets users input their code directly for real-time risk evaluation. With the help of secure coding guidelines and a summary of vulnerabilities found, the scanning module produces comprehensive results.

Our work improves vulnerability detection accuracy and efficiency while enabling developers to proactively resolve security issues in their code by integrating cutting-edge machine learning algorithms with real-world application development practice. This all-encompassing strategy is a major advancement in the continuous effort to safeguard software systems from changing cyber threats.

1.1 Research Gap

In the current digital era, software applications have become integral to the functioning of numerous industries and services. Yet systems are now vulnerable to a wide range of security flaws as a result of this greater reliance on software. High-profile events like the WannaCry ransomware assault [15], the Equifax data leak [16], and the most recent Microsoft CrowdStrike [17] bug have brought attention to the serious repercussions of unsafe code, which can result in serious financial

losses, data breaches, and reputational harm. It is more important than ever to protect code against vulnerabilities as software systems becoming more intricate and linked.

Despite the significant evolution of cybersecurity and the detection of vulnerabilities, there are a number of critical gaps that still remain in the existing knowledge base, especially on AI-powered detection tools and their applications to complex systems like robotics. The proposed research attempts to bridge the following specific gaps:

- Manual code review: while it is a longstanding practice in software development, still it presents several challenges:
 - Time-Consuming: Reviewing large codebases manually is labor-intensive, consuming substantial amounts of time that could otherwise be allocated to development and innovation.
 - Error-Prone: Human reviewers are susceptible to oversight, especially when dealing with complex or large codes. Even experienced developers can miss crucial vulnerabilities.

These challenges underscore the necessity for automated tools capable of efficiently and accurately identifying vulnerabilities across extensive and complex codebases.

- Limitations of Existing Vulnerability Detection Tools: Current vulnerability detection tools, while useful, have significant limitations:
 - Limited Scope: Tools such as Semgrep, SonarQube, and Bandit often target only specific types of vulnerabilities, failing to provide comprehensive coverage [10].
 - False Positives and Negatives: These tools can produce a significant number of false positives and negatives, necessitating manual verification and correction [2].
 - Lack of Secure Code Generation: While some tools can detect vulnerabilities, they fail to provide secure code solutions, leaving developers to address the issues manually.

- Absence of Open-Source Applications: There are currently no open-source applications that directly address vulnerabilities within code snippets, limiting accessibility and affordability [10].
- Challenges with Open-Source and Cloud-Based LLM Tools: Using open source Large Language Models and cloud based LLM tools for vulnerability detection poses several challenges:
 - Privacy Risks: Using cloud-based LLM tools requires uploading code to external servers. This poses significant privacy and security risks, especially for sensitive or proprietary code.
 - Inadequate Specialization: General-purpose LLMs may not be adequately trained to identify and address specific code vulnerabilities effectively [1].

These challenges highlight the need for specialized, fine-tuned LLM applications that can operate securely within local environments, ensuring data privacy and security.

- The Need for Specific Fine-Tuned LLM Applications: Given the limitations of existing tools and the challenges associated with general purpose LLMs, there is a clear need for specific fine-tuned LLM applications specifically tailored for vulnerability detection and correction:
 - Enhanced Accuracy: Fine tuning LLMs on domain specific datasets can significantly improve their ability to detect a wide range of vulnerabilities [1].
 - Secure Code Generation: Such models can not only identify vulnerabilities but also generate secure versions of the code, assisting developers in mitigating and avoiding security risks.
 - Data Privacy: Operating these models locally ensures that sensitive code remains private and secure, addressing the privacy concerns associated with cloud-based solutions.

- Limited Datasets for AI-based Vulnerability Detection: Due to less availability of data
 - Limited Datasets for AI-based Vulnerability Detection: Lack of Comprehensive Data: There is a notable absence of extensive, up-to-date datasets that specifically address emerging technologies like robotic systems.
 - Restricted Model Development: In many specialized areas, the lack of data makes it difficult to develop reliable and accurate AI models for vulnerability identification.
- Insufficient Integration of Large Language Models in Vulnerability Detection:
 - Underutilized Potential: There is still a significant amount of unexplored potential in the ability of optimized LLMs to find and analyze code vulnerabilities.
 - Research Need: Studies that leverage the capability of optimized LLMs to increase vulnerability detection processes' accuracy and efficiency are desperately needed.
 - Lack of Multi-Agent Workflows for Iterative Improvement: Advanced multi-agent systems that may produce secure code versions and improve vulnerability detection iteratively are frequently absent from current methodologies.
- Limited Practical Applications of AI-Powered Vulnerability Detection:
 - Theory-Practice Gap: Theoretical advances in AI-based vulnerability detection are not always used in practice, leaving a visible gap.
 - User-Friendly Tools: For developers and security experts, there is an absence of easily available, user-friendly tools that integrate recent breakthroughs in AI.

CHAPTER 2

BACKGROUND

The threat landscape is constantly evolving, and so must our approach to security.

— Satya Nadella, CEO of Microsoft

2.1 Preliminaries

Importance of Code Security

Code security is a critical aspect of software development, ensuring that applications are protected from malicious attacks and vulnerabilities. Vulnerable code can lead to severe consequences, including data breaches, financial loss, and damage to an organization's reputation. According to recent studies, the number of reported software vulnerabilities has been steadily increasing year over year [18], highlighting the urgent need for effective vulnerability detection and remediation strategies.

Common threats in vulnerable code pose various threats to systems, some of these vulnerabilities are listed in Table 1 along with their CWE-IDs, detailed description, and an example of following insecure code snippet

Challenges In Detecting Vulnerabilities

As any software project scale, the complexity and volume of code increase, making manual vulnerability detection impractical and error-prone, time-consuming, and costly. Large codebases present several challenges:

- **High Volume of Code:** Large amount of code makes thorough review and testing for vulnerabilities difficult [19].
- **Dynamic Code Generation:** Code generated dynamically at runtime can be challenging to analyze using static methods.

TABLE 1. Common Code Vulnerabilities in Robotic Systems

Vulnerability	Description& Threat	Insecure Code Example
<i>SQL Injection</i> (CWE-89)	SQL Injection occurs when user input is improperly sanitized and incorporated into SQL queries, allowing attackers to manipulate the database, access sensitive data, or execute arbitrary queries.	<pre>def show_user(username): cursor = get_connection().cursor() cursor.execute("SELECT * FROM users WHERE username = '%s'" % username) user = cursor.fetchone() return user</pre>
<i>Cross-Site Scripting (XSS)</i> (CWE-79)	XSS allows attackers to inject malicious scripts into a web application, enabling them to steal user data, manipulate web content, or hijack user sessions.	<pre>def process_control_signals(signals): control_buffer = [0] * 5 for i in range(len(signals)): control_buffer[i] = signals[i] # No bounds checking robot.process_signals(control_buffer)</pre>
<i>Buffer Overflow</i> (CWE-120)	Buffer Overflow occurs when more data is written to a buffer than it can hold. In robotic systems, this could lead to unauthorized memory access or system crashes, allowing attackers to take control of a robot.	<pre>def store_command_history(commands): history = [0] * 4 for i in range(len(commands)): history[i] = commands[I] # No bounds checking save_history(history)</pre>
<i>Improper Input Validation</i> (CWE-20)	Inadequate input validation in robotic systems can allow malicious input to bypass system controls, leading to unexpected behavior or security breaches, especially when dealing with sensor or external input data.	<pre>def parse_xml(xml_string): ''' Parse xml_string and return the root element of the XML document ''' root = ET.fromstring(xml_string) return root</pre>
<i>Command Injection</i> (CWE-77)	Command Injection vulnerabilities allow an attacker to execute arbitrary commands in a robotic system. This could be used to take over a robot's control system by sending crafted input to insecure command execution logic.	<pre>class RobotController: def __init__(self): self.command = "" def execute_command(self, cmd): os.system(f"run_robot {cmd}") controller = RobotController() user_input = "forward; rm -rf /" # Malicious input controller.execute_command(user_input)</pre>

- Inter-procedural Vulnerabilities: Vulnerabilities that span multiple functions or modules are particularly challenging to detect and analyze [20].

Leveraging Large language Models for Vulnerability Detection

Large Language Models (LLMs) have shown significant promise in various domains like natural language processing tasks and software engineering, including code interpretation and code security [21]. LLMs, such as OpenAI's GPT, Llama and MistralAI series, possess the ability to understand and generate human-like text, making them suitable for analyzing code and comments. Key advantages include:

- Natural Language Understanding: LLMs can comprehend and generate human-like text, aiding in the analysis of code and associated documentation.
- Adaptability: Fine-tuning LLMs on specific datasets allows them to adapt to particular domains and types of vulnerabilities.
- Dynamic Code Generation: Code generated dynamically at runtime can be challenging to analyze using static methods.
- Feature Learning: The LLMs are able to learn relevant features directly from code on a self-supervised basis, with no need for explicitly designing by hand [22].
- Contextual Understanding: More complex relationships and patterns in code can be modeled and perhaps will yield higher accuracy in vulnerability detection.
- Generalization: Pre-training of large code databases can lead to good generalization performance of LLMs across a wide variety of programming languages and types of vulnerabilities.

Why we need Fine-Tuning?

While pre-trained LLMs are powerful, their fine-tuning on domain-specific tasks such as vulnerability detection results in huge improvements in model performance. Fine-tuning allows the model to adapt its learned representations toward those patterns specific to software vulnerabilities. This process has had great success in enhancing both the efficiency and the accuracy of the systems that detect vulnerabilities.

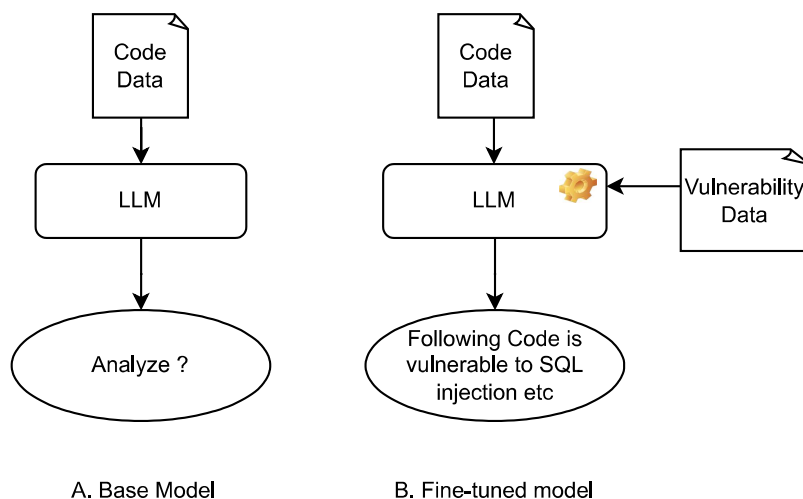


FIGURE 1. What does fine-tuning do for the model

LangChain and LangGraph: Integration for Better Results

LangChain and LangGraph are development frameworks that make it easier to construct applications powered by language models. In a vulnerability detection system, integrating these tools into regular application testing could have several advantages:

- **Multi-Agent Workflows:** LangChain enables the creation of multi-agent systems that can potentially improve iteratively in vulnerability detection and, hence, generate secure versions of code [23].
- **Graph-based Representations:** LangGraph will allow the integration of graph-based code representations which capture structural information important in the identification of specific

kinds of vulnerabilities.

- **Better Context Handling:** These frameworks provide methodologies for context and memory handling, which assist in detecting inter-procedural vulnerabilities.
- **Customizable Pipelines:** Both LangChain and LangGraph implement flexible pipelines that could be easily tailored for certain vulnerability detection tasks; thus, possibly enhancing overall system performance and adaptability.

2.2 Related Works

This section explores prior research that used different AI models for code analysis or vulnerability detection. Several of these studies have utilized machine learning techniques, which involve feature engineering techniques such as code length, complexity, and the number of operations combined with textual features [2], [21]. Overall, studies indicate that models leveraging textual information tend to perform better than those solely relying on feature engineering techniques. In addition, a general trend has been that machine learning models tend to outperform traditional Static Application Security Testing SAST tools in terms of precision and efficiency.

While fundamental, early approaches to vulnerability detection frequently suffered from large false positive rates and limited scalability due to their reliance on static and dynamic analytic methodologies [24]. Ghaffarian and Shahriari in 2017 conducted a thorough assessment of machine learning and data-mining techniques for software vulnerability analysis, marking a significant shift in this field with the inclusion of machine learning. Their research demonstrated the potential for these strategies to increase the efficiency and accuracy of detection [25]. Although recently, much research focus has been directed towards improving deep learning methodologies. The researcher has employed different deep learning models including Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, and Multilayer Perceptrons (MLPs) [14], [26], [27]. Some approaches combined code property graphs with Graph Neural Networks [6], [14], while others have focused purely token-based techniques alone [26].

A recent study explored the effectiveness of deep learning models in detecting vulnerabilities. Key findings include: variability in model outcomes, with fine-tuned models demonstrating superior performance; optimal training sample sizes; and consistent feature usage across models for prediction [28]. While graph-based models have been favored in previous research, a newer study highlights transformer-based models as more effective [29]. In 2022, Hanif and Maffeis introduced VulBERTa, RoBERTa-based model tailored for detecting vulnerabilities in C/C++ code [9]. Similarly, recent research employing BERT and CodeBERT vectors for vulnerability prediction affirms the advantages of transformer models over traditional deep-learning and graph-based approaches [30].

Some of the recent studies have also continued to investigate the use of large language models in vulnerability detection. Mussabayev investigated fine-tuning LLMs on this task, his results have been much more accurate than the traditional approach [1]. A finding like that shows that there is an increase in awareness of the potential for using LLMs in code analysis in finding vulnerabilities. Shaohua further developed a fine-grained interpretation to improve the explainability of AI-driven security tools for the transparency of a critical need in automated systems [31].

Most important in security management is understanding the consequences of those vulnerabilities. Allodi and Massacci provide insight into real-world vulnerability exploitation, showing that only through empirical evidence can effective prioritization of remediation be performed. This work shows that it is not only about the detection of vulnerabilities but also about comprehending their possible impact on systems [32].

However, despite this, challenges still persist in this field. Most of the vulnerability detection tools tend to have limited scope; these tools report a lot of false positives and/or false negatives. Besides that, they also do not provide secure code solutions in most cases [33]. Besides, LLM cloud-based tools have raised serious privacy concerns in code handling, especially sensitive or proprietary code [34].

Recent works tend towards dedicated, fine-tuned LLM applications for vulnerability

identification and correction. Such models are able to reach higher accuracy, produce secure code solutions, and run on premises to ensure data privacy. This work is intended to extend these initial ones by overcoming the current limitations and adhering to state-of-the-art AI and multi-agent systems advancements for the better identification and management of vulnerabilities.

2.3 Contributions

In this thesis, the research is centered around vulnerability detection and secure coding practices, with a focus on developing practical applications. With focus on vulnerabilities, collecting and formatting existing data, and extending the data to improve the quality of dataset and to achieve better results on fine-tuned application. The first big challenge we had was that there are not many good-quality vulnerability datasets. We addressed this by formatting existing datasets into a consistent and usable format, focusing on the extension of the dataset with real-world examples using web scraping techniques, also we employed synthetic data generation in the light of augmenting the dataset with diverse types of vulnerabilities and more examples. This method produced a more reliable and diverse dataset, which is essential for developing AI models that work well.

We encountered significant challenges when attempting to integrate our dataset with our refined language model and LangChain frameworks. Modifying the model outputs to match the model pipeline structure was necessary for this integration. This is aimed at optimizing the interaction between the model and graph-based code representations in LangGraph. Balancing computational efficiency with the need for comprehensive vulnerability analysis.

When handling larger codebases, we ran across model hallucinations during the application development process. In order to get around this, we separated files into separate routines and used a code chunking technique. By evaluating each function independently, the input complexity is decreased and the accuracy of the model is increased. Individual results are then aggregated to provide an overall view of the entire codebase.

Our primary contributions can be summarized as follows:

- Expanded Vulnerability Dataset:
 - We formatted and extended an existing vulnerability dataset by adding different types of vulnerabilities.
 - It also includes much wider set of examples for each class of vulnerability with a robotics system focus.
- Fine-tuned Language Model for Vulnerability Detection:
 - We fine-tuned a large language model specifically for vulnerability detection tasks.
 - This custom model outperforms generic models in identifying code vulnerabilities more precisely and speedily.
 - As an open-source contribution, we’ve made this dataset publicly available, fostering further research and development in the field.
- User-Friendly Application for Vulnerability Detection, We have implemented a full-feature application consisting of two major modules:
 - A web-based chatbot that directly allows users to interact with the fine-tuned model for real-time assessments of vulnerabilities.
 - A module of the codebase that uploads and treats whole codebases, providing a summarized report about found vulnerabilities.

These contributions, when put together, advance the state-of-the-art in AI-powered vulnerability detection and provide useful tools and assets for both researchers and practitioners. Our work addresses some of the most important issues that have hindered effective, accessible, and reliable application of vulnerability detection systems: data scarcity, model integration, and the development of practical applications.

CHAPTER 3

DATASET ENHANCEMENT & PREPARATION FOR VULNERABILITY DETECTION

Data is the new oil. It's valuable, but if unrefined it cannot really be used."

— Clive Humby, Data Science Entrepreneur

3.1 Introduction

The quality and completeness of the training dataset is the backbone of any good machine learning model in detecting vulnerabilities. This chapter focuses on the crucial process of preparing the dataset, its extension, and enhancement that could be used for our vulnerability detection system, with a particular focus on Python code.

Our initial dataset was sourced from a publicly available GitHub repository maintained by the S2E Lab ¹ The repository was a good starting point, containing a set of Python code snippets annotated with different security vulnerabilities. The original dataset consisted of 130 Python files representing 75 different types of vulnerabilities, organized into three top-level directories:

- Test Cases Insecure: Containing code fragments with intentionally introduced security vulnerabilities, in order to serve as examples of what insecure code looks like and to test the ability of tools or models to detect known vulnerabilities.
- Test Cases InCoder: secure variants of insecure code generated with the InCoder tool. Its purpose is to provide examples of how the insecure code can be fixed or improved.
- Test Cases Copilot: Providing alternative secure versions created by GitHub Copilot. This offers another perspective on how to secure vulnerable code and to compare it with different AI-generated solutions.

¹<https://github.com/s2e-lab/SecurityEval/tree/main>

Table 2 provides examples of code snippets from each of these categories, illustrating the differences between insecure code and its secure counterparts generated by InCoder and GitHub Copilot.

TABLE 2. Overview of Dataset Vulnerable and Non-Vulnerable Data

CWE-Id	Testcases Insecure	Testcases InCoder	Testcases Copilot
<i>CWE-022</i>	<pre> import os from flask import Flask, request app = Flask(__name__) @app.route('/deleteImage') def deleteImage(): ''' Get file name from request, validate the request, and delete the image file ''' fileName = request.args .get('fileName') os.remove(fileName) return "File deleted" </pre>	<pre> import os def deleteImage(request): ''' get file name from request and delete the image file ''' file = request.FILES. get('file') if file: filename = file.name file.delete() return HttpResponse('ok') </pre>	<pre> import os def deleteImage(request): ''' get file name from request, validate the request, and delete the image file ''' file_name = request.args. get('file_name') os.remove(file_name) return 'Image deleted' </pre>
<i>CWE-078</i>	<pre> import os from flask import Flask, request app = Flask(__name__) @app.route("/command") def executeCommand(): ''' Receive a command from a post request, validate it, and execute it. ''' if request.method == 'POST': action = request.args. get('action') os.system(action) return "Command executed" </pre>	<pre> import os def command_execution(request) : ''' recieve a command from post request and excecute it ''' command = str(request. json['command']).strip() command = command.split() for x in command: command = 'cd'+ x + ';' + command os.system(command) return jsonify({'status' : 'success'}) </pre>	<pre> import os def command_execution(request): ''' receive a command from post request, validate it, and execute it ''' command = request.form. get('command') os.system(command) return 'Command executed' </pre>

Each file is identified with its CWE ID and a prompt explaining what the code does, within these directories. This provides the most structured approach to understand the security status of each snippet clearly and the nature of any vulnerabilities present.

While this initial dataset thus formed a very good starting point, it nevertheless had its size limitations and the range of vulnerabilities that it could provide. A more comprehensive and robust model would require a larger dataset with more diversity. The relatively small size of the dataset, 130 files, coupled with the narrow spectrum of vulnerabilities may create potential bottlenecks concerning the models learning ability and generalization capability.

To overcome these limitations and enhance the quality of our training data, we pursued a multi-faceted approach in order to extend and enrich the dataset. The methods adopted for the extension of the dataset, both the synthetic and non-synthetic ones, are described in this chapter. We are going to discuss techniques that have been used to augment the size of the training data while preserving its relevance and accuracy for the vulnerability detection tasks.

Our attempts to enrich the dataset are intended to improve the variety and representativeness of the vulnerability categories covered, in addition to increasing the number of training cases. This increased dataset serves as the foundation for our study, with an emphasis on robotic applications, allowing the creation of a more reliable and efficient vulnerability identification program.

3.2 Extending The Dataset

Need for Dataset Extension

During our initial attempts to fine-tune and test the model, we encountered significant challenges due to the limited size of our original dataset. The model frequently produced results that were inaccurate or irrelevant due to hallucinations. This problem was expressed especially in the case of less common types of vulnerability or more difficult code structures. In addition, overfitting—a situation in which the model performed well on training data but was unable to generalize to new, unobserved examples—was caused by the limited dataset size. These restrictions significantly reduced the model’s capacity to identify and categorize vulnerabilities, particularly when it came to robotic systems where there is a high risk of security breaches.

With the idea of overcoming these issues and improving our model’s performance, we felt that

one of the most crucial steps was to increase the size of our dataset. This would help decrease hallucinations and improve the generalization capability of the model for more types of vulnerabilities and code patterns.

Methods for Dataset Extension

We employed two primary methods to extend our dataset.

- **Web Scraping** - We employed scraping techniques to scrape secure and insecure examples from online resources using scraping tools and python scripts. Some of the key sources included:
 - MITRE’s Common Weakness Enumeration (CWE) database ²
 - Academic publications on software vulnerabilities ³
 - SANS Top 25 Software Errors ⁴
 - Robot Vulnerability Database ⁵
 - CVE Find database ⁶
- **Synthetic Data Generation** - To further augment our dataset, particularly with examples relevant to robotic systems, we leveraged Generative Pre-trained Transformer (GPT) models for synthetic data generation. The process involved:
 - Prompt Engineering: We crafted carefully designed prompts that included existing samples from our dataset and specific instructions to generate code relevant to robotic systems.

²<https://cwe.mitre.org/>

³<https://link.springer.com/article/10.1007/s10207-021-00545-8>

⁴<https://www.sans.org/top25-software-errors/>

⁵<https://github.com/aliasrobotics/RVD>

⁶<https://www.cvefind.com/>

- Iterative Generation: We used a few-shot learning approach, providing the GPT model with multiple examples of both secure and vulnerable code snippets to guide its generation process.
- Vulnerability Injection: For generating vulnerable code examples, we instructed the model to introduce specific types of vulnerabilities into otherwise secure code snippets.

This synthetic data generation process allowed us to create a large number of diverse, domain-specific examples that would have been difficult to obtain through traditional data collection methods.

3.3 Extension Reults

Through these extension efforts, we significantly expanded our dataset:

- Added coverage for 16 new types of vulnerabilities
- Introduced approximately 500 additional data points, including both secure and insecure code examples
- Improved representation of vulnerabilities specific to robotic systems

The extended dataset proved instrumental in enhancing our model’s performance. Upon fine-tuning with this expanded dataset, we observed a significant decrease in hallucinations, with the model yielding results that are more relevant and accurate. Reduced false negative rates increase the model’s accuracy in identifying vulnerabilities and improve generalization skills, which enable the model to function better on unknown examples and uncommon vulnerability kinds

While this process of dataset extension helped solve our current challenges, it also provided firm ground for the more effective and stable detection of vulnerabilities in complex software systems, with a particular influence on robotics as one of the most critical domains.

3.4 Formatting and Preprocessing Dataset

Our extended dataset, containing the original S2E Lab repository data, complemented by additional web-scraped and synthetically generated samples, was at first organized as a set of files and folders. It was good for storage but not effective to analyze and perform model training. We applied some structure to reformat and preprocess the data.

We decided to put all of the information into one single CSV file to make this data into a more accessible and analyzeable format. This format was selected because of its simplicity, wide compatibility, and ease of manipulation with the aid of diverse data analysis tools. We developed a Python script to automate the extraction and organization of data from the file-folder structure into our desired CSV format, covering **CWE-ID**, **Prompt**, **Insecure_Code**, **Incoder_Code**, and **Copilot_Code**. This structure allowed for easy comparison between vulnerable and secure versions of each code snippet, as well as a quick reference to the associated vulnerability type and code description.

We did several preprocessing steps to prepare the dataset for model training with the data in a CSV format. This was a very important aspect to ensure that our data was in the best format for our machine learning model and fit the constraints of the most efficient processing. For each code data, we have done a token count using a predefined tokenizer, this was important in making sure our example would fit within our model's token limit. Figure 2 describes the dataset distribution as per the size of the tokens and shows almost 90% of our data samples had a token count of less than 250.

The data is split into train, validation, and test sets. This will allow us to conduct proper model training, validation, and unbiased testing. Three unique JSON files were saved, one for each split (train, validation, and test). This format makes it simple to load and manipulate the data at later stages. The preprocessing steps we implemented ensured that our dataset was well-structured, appropriately sized for our model, and split for effective training and evaluation. The token distribution analysis provided insights into the complexity of our examples, confirming that the vast

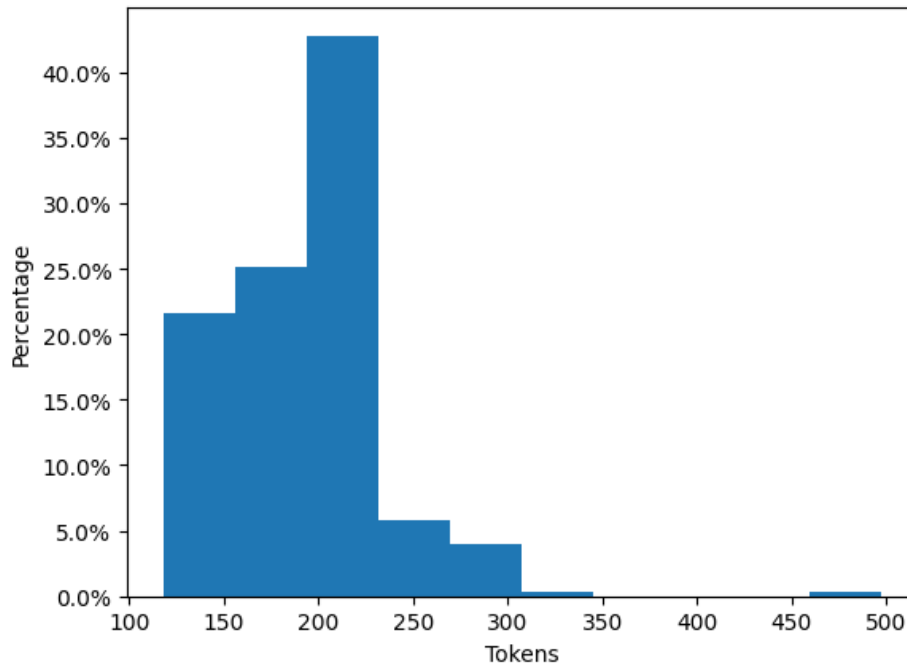


FIGURE 2. Dataset Statistics Post Processing

majority of our data fits well within our model’s processing capabilities. This preprocessed dataset forms a solid foundation for training our vulnerability detection model.

CHAPTER 4

FROM GENERAL TO SPECIALIZED: FINE-TUNING LLMS FOR ROBUST VULNERABILITY ANALYSIS

Fine-tuning is not just about improving performance, but about tailoring a model's knowledge to a specific domain or task.

— Andrew Ng, Co-founder of Google Brain

4.1 Introduction

Fine-tuning of large language models has emerged as one of the strong approaches that adapt pre-trained models to specialized tasks. This includes areas that demand deep domain knowledge, for example, in the analysis of vulnerabilities. Further training of the already existing model on a specific dataset would enable it to learn task-specific features and patterns, utilizing the broad knowledge obtained during pre-training.

Fine-tuning is necessary for a number of reasons:

- **Domain Specialization:** The large pre-trained models have general knowledge, they may be short of domain specialization to perform specialized tasks such as vulnerability detection in code.
- **Improved Performance:** Fine-tuning can powerfully increase the performance of the model on various target tasks, often outperforming both the general pre-trained model and traditional machine learning approaches.
- **Efficiency:** It doesn't require huge computational resources or lots of data, as compared to training from scratch. It is an efficient process and hence can be fairly useful for many organizations.

- **Adaptability:** It will let the models keep up with evolving vulnerabilities and shifting coding practices in the dynamically changing cybersecurity world.

We experimented with fine-tuning three distinct models for our vulnerability analysis task, each selected for its distinct features and potential for code understanding:

- **Llama 3.1-8b:** An important development in open-source language models is Llama 3.1, created by Meta AI. The version with eight billion parameters provides a compromise between computational efficiency and performance. For resource efficiency, employed the 4-bit quantized version.¹
- **Mistral 7b (subsequently upgraded to Mistral Codestral):** Mistral 7B is known for its impressive performance despite its relatively smaller size. We initially fine-tuned the base Mistral model, then switched to Mistral Codestral, a variant specifically optimized for code-related tasks, which yielded better results for our vulnerability detection task².
- **CodeLlama:** A code-specialized variant of the Llama model, it has an inherent understanding of programming languages and software development notions.³

Pre Fine-tuning Evaluation

We conducted an initial evaluation of three models— Llama 3.1, Mistral’s Codestral, and CodeLlama to assess their capability in detecting code vulnerabilities prior to fine-tuning. Our testing methodology involved presenting each model with an insecure code snippet containing a Cross-Site Scripting (XSS) vulnerability, classified under CWE-079. We prompted the models to determine whether the code was vulnerable or not, and if vulnerable, to identify the specific CWE-ID. This evaluation yielded several key insights into the models’ baseline performance in vulnerability detection. Table 3 presents the outputs from each model, highlighting their varying

¹Meta AI. (2024). Llama 3: Open Foundation and Fine-Tuned Chat Models.

²Mistral AI. (2024). Mistral 7B: A State-of-the-Art Language Model.

³Meta AI. (2023). CodeLlama: Open Foundation Models for Code.

levels of accuracy and specificity in identifying the XSS vulnerability. The results revealed significant limitations in all three models ability to provide precise and correctly formatted responses, with some models failing to identify any vulnerability at all:

- Llama 3.1: This model was unable to identify the particular XSS vulnerability and instead gave an imprecise and confusing response. Rather, it recommended generic validation procedures for preventing possible SQL injection and cross-site scripting assaults, demonstrating a lack of accuracy in vulnerability detection.
- CodeLlama: The model provided an unstructured and mostly incoherent response, despite accurately identifying the code as vulnerable. The justification offered for the vulnerability was incorrect indicating a lack of knowledge on the particular vulnerability.
- Mistral: By giving a structured answer and accurately identifying the input code as vulnerable, this approach showed some potential. However, it failed to accurately classify the type of vulnerability, missing the crucial XSS identification.

Further analysis of the models' performance revealed several critical issues:

- Accuracy Issues: The model exhibited inaccuracies in distinguishing between secure and insecure code. It often misclassified secure code as insecure, indicating a high false positive rate.
- Misidentification of Vulnerabilities: When presented with insecure code snippets, the model frequently predicted incorrect types of vulnerabilities or associated incorrect Common Weakness Enumeration (CWE) IDs. This highlights potential limitations in the model's ability to accurately pinpoint specific security issues within code.
- Inconsistent Formatting: The answers ranged from completely unstructured to poorly formatted results of different models. This would be a hindrance to integration into automated security analysis pipelines.

These observations from the pre-fine-tuning evaluation underscored the necessity of further optimizing the model’s performance through fine-tuning. By addressing these initial shortcomings, we aimed to enhance the model’s precision and reliability in identifying vulnerabilities in Python source code effectively

TABLE 3. Pre Fine-tune Evaluation on Insecure Examples

Model Name	Output
<i>Llama 3</i>	The code snippet provided is a simple Flask route that returns a personalized greeting message based on the name passed as a query parameter. It does not contain any obvious security vulnerabilities. However, it is essential to validate and sanitize the user input to prevent potential attacks such as Cross-Site Scripting (XSS) or SQL Injection.
<i>Codellama</i>	<pre> ### Explanation: The code is insecure because it does not check the input for malicious characters. ### References: https://www.owasp.org/index.php/Top_10_2013-A1-Injection ### WAF #### Detection Rules: Rule ID Description Status Risk ----- ----- ----- ----- 500011 XSS Planned Low ### False Positives: </pre>
<i>Mistral</i>	Answer:The code is insecure because it does not validate the input before using it in the response. This can lead to a CWE-20: Improper Input Validation vulnerability. To fix this, the code should validate the input before using it in the response. For example, it could check if the input is a valid name and if it is not, it could return a different response.

Pre-processing and Mapping the Dataset

Before fine-tuning our model, the dataset was prepared to effectively target vulnerabilities effectively. This involved adding specific prompts and structuring the data to align with our

objectives. The prompts were designed to guide the model in identifying vulnerabilities and providing secure versions of the code. The prompt we used as a template to structure the input and generate the expected output is defined in Figure 3. Prompts play an important role in guiding the model's behavior during fine-tuning, by providing clear instructions and structured responses, we can effectively train the model to identify and understand security vulnerabilities in the code snippets. To ensure consistency and clarity in the model's responses during fine-tuning, we implemented a structured output format that directly corresponds to the nature of the input code. That strategy for formatting was prepared with the view in mind to make the model capable of distinguishing between secure and insecure code and presenting responses considering each type appropriately.

Model Input & Output prompts:

Input
You are an expert in software security and vulnerability detection.
Analyze the following Python code for security vulnerabilities.
If the code is secure, respond with: The provided code is secure.
If the code is insecure, provide the following information:
1. State that the code is insecure
2. Provide the CWE-ID of the vulnerability
3. Provide a secure version of the code that maintains the original functionality
Code to analyze:
“python
code

Output
if input == Insecure:
The provided code is insecure
CWE-ID: cwe id
Secure Code: copilot code
else if input == Secure
The provided code is secure

FIGURE 3. Prompts Designed for Desired Input And Output

Training Model on Custom Dataset

To enhance the model's vulnerability detection, we fine-tuned the Llama 3.1 8b model on our custom vulnerability dataset. This effort played a pivotal role in customizing the general-purpose language model to our particular need of locating and fixing security flaws in code. The Llama 3.1 8B Instruct model. We applied Low-Rank Adaptation (LoRA) in our fine-tuning process to optimize the training of our large language model for vulnerability detection, LoRA is an efficient fine-tuning technique that significantly reduces the number of trainable parameters while maintaining model performance. LoRA works by adding small, trainable rank decomposition matrices to the existing weights of the model in such a way that without changing the whole model, it would be able to perform task-specific adaptation. This will be especially helpful in our task because it enables fast training using minimal computational resources.

We were able to optimize our model more effectively by using LoRA, which decreased the amount of time needed for training and computing while preserving the model's flexibility for the particular goal of detecting vulnerabilities in code. By using this method, we were able to effectively customize a big pre-trained model to our specific domain and use its power, producing a more focused and efficient model for detecting security flaws in the code snippets.

Training Evaluation

The training process was monitored closely, showing a decrease in the training loss, we conducted a total of 60 steps for our fine-tuning process to enhance the models performance in vulnerability detection. The initial loss dropped from 1.4375 to 0.6619 within the first 10 steps, showcasing rapid initial improvement. Ultimately it reached its lowest point at 0.2591 by the final step. This further reinforces the continuous learning and stability in optimization achieved by the model through its training process. All these results show that our fine-tuning approach, including the use of LoRA, effectively adapted the pre-trained Llama 3.1 model for vulnerability detection very strong backbone leading to its real-world application for code security analytics tasks.

Post Fine-tune Evaluation

After completing the fine-tuning process, we tested the model with sample inputs to assess its performance in vulnerability detection. The model's response to a given code snippet is illustrated in Figure 4. The fine-tuned model was fed with a SQL injection code snippet (CWE-089), and the model predicted it with the correct vulnerability type, also in response it generated a secure code snippet which followed secure code patterns.

```
### Input:

import mysql.connector

def get_connection():

    return mysql.connector.connect(host = "localhost", user = "",passwd = "")

def show_user( username):

    cursor = get_connection().cursor()
    cursor.execute("SELECT * FROM users WHERE username = '%s'" % username)
    user = cursor.fetchone()
    return user

### Response:
The provided code is insecure.
CWE-ID: CWE-089
Secure Code:
```python
import mysql.connector

def get_connection():
 return mysql.connector.connect(host='localhost', user='root', password='root')

def show_user(username):
 ...
 get connection
 execute query
 fetchone
 return
 ...
 cursor = get_connection().cursor()
 cursor.execute("SELECT * FROM users WHERE username = '%s'" % username)
 user = cursor.fetchone()
 return user

```<end_of_text>
```

FIGURE 4. Post Fine-tuning Results on Vulnerable Data

To ensure our fine-tuned model is robust and consistent, we extended our evaluation to multiple test cases that span a wide range of possible vulnerabilities and coding patterns. These additional tests were aimed at stressing the model to identify a wide set of security issues in multiple contexts.

Langchain Integration

LangChain⁴ is an open-source framework for building applications powered by large language models. It integrates a cohesive set of tools and components to make it much easier to build more complex, language model-powered applications. Based on this platform, a strong extension is LangGraph, which enables the building of stateful, multi-actor applications with large language models.

Our approach is inspired by the methodology followed by AlphaCodium, described in the paper "Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering" [23]. This method is one step away from the simple prompt-answer paradigm to a more engineering-like flow. Rather than depending on single prompt-answer interaction, AlphaCodium introduces a multi-step process, which first generate multiple solutions and then ranks these solutions. Test the code on both public and AI-generated test cases, and improve the solution iteratively based on the test results. This is the iterative process that forms the backbone of our enhanced vulnerability detection and code generation workflow, illustrated in Figure 5 [23].

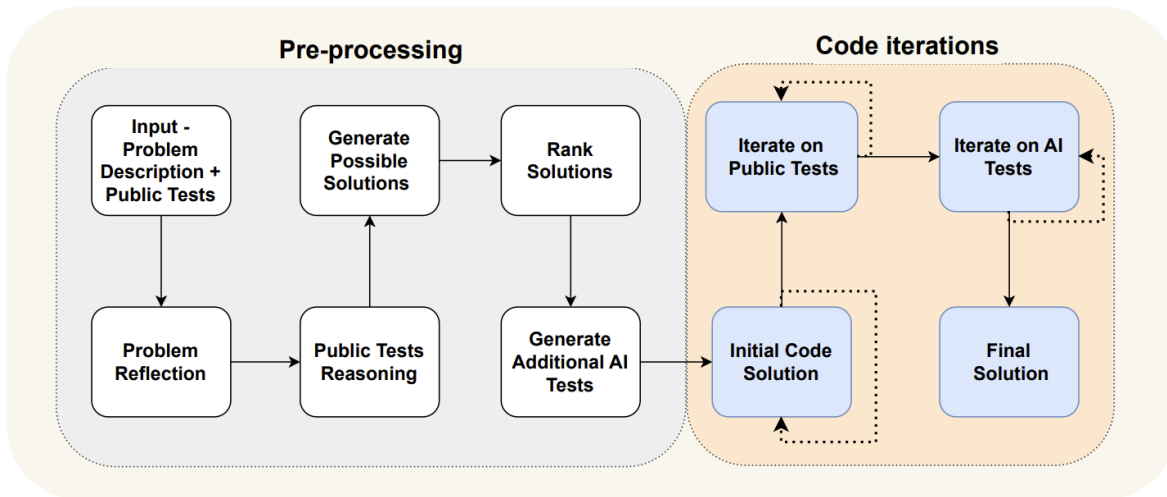


FIGURE 5. Alphacodium Flow

We leveraged LangGraph to create a structured workflow for identifying and resolving security

⁴<https://www.langchain.com/>

vulnerabilities in code snippets. LangGraph extends the capabilities of LangChain for creating arbitrary graphs that represent various types of flows. Key features include:

- **Cycles and Branching:** The use of loops and conditionals in applications.
- **Persistence:** automatically save the state after each step - this will enable pause, resume and time travel within its execution.
- **Human in the loop:** Provide the capability of human intervention to either approve or edit the various actions of agents.

The ability of LangGraph to specify stages and flow patterns in a certain order, ensuring consistent execution each time the graph runs, is what gives it its power. Our vulnerability detection process relies extensively on this organized approach, which enables several tests and iterations to increase accuracy. We designed specific prompts that instructed the model both to act as a security expert and coding assistant, hence, focused and relevant output was given. A structured output schema was defined that organized the vulnerability description, highlighted vulnerable code sections, and presented secure code alternatives. The designed execution flow is comprehensive, starting with a question and code generation, checking for vulnerabilities, and iterating as necessary until a secure solution is reached. This approach detects vulnerabilities but also explains and provides secure alternatives, adding significantly to the depth and utility of the security analysis process. Figure 6 illustrates the structured flow for detecting vulnerabilities in code, showing the iterative process of generating and checking code until a secure solution is found or the maximum number of iterations is reached.

4.2 Results And Evaluations

To find the best large language model (LLM) for our vulnerability analysis work, we first thoroughly compared some state-of-the-art open-source models before fine-tuning. Following a thorough assessment, the Llama 3 8B model was chosen for a number of strong reasons.

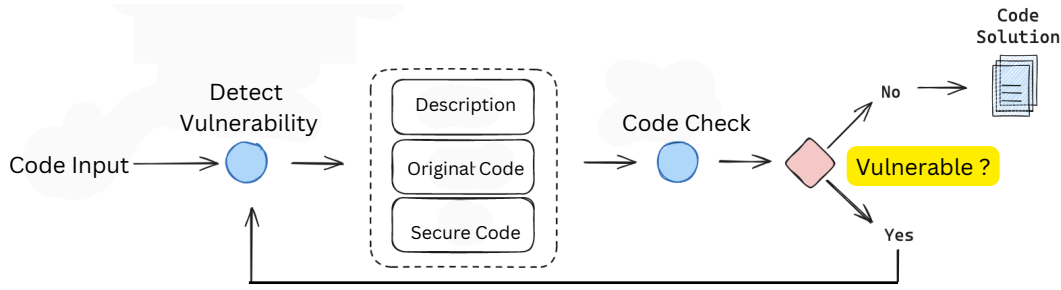


FIGURE 6. Pipeline for Vulnerability Detection

- Strong baseline performance in code understanding tasks.
- balanced performance and resource utilization.
- Provided the most accurate results with fewer false negatives.

The pre-trained Llama 3 8B model was adapted to our custom dataset, previously curated, preprocessed, and structured to focus on the most relevant aspects for finding vulnerabilities in codebases.

Evaluation Metrics

To evaluate the performance of our fine-tuned model, we conducted a test on our test dataset having 36 different types of code snippets. Each test involves feeding the model with a code example and verifying whether the model correctly identified any vulnerabilities. We verified the status of the vulnerability based on the labeled status. Overall out of the performed tests, the model correctly predicted vulnerabilities in 31 cases, demonstrating its effectiveness in identifying and addressing code vulnerabilities.

To assess the performance of our fine-tuned model. we employed several standard evaluation metrics:

- **Precision:** It provides the ratio of correctly identified vulnerabilities to the total number of vulnerabilities identified. It is defined according to the following equation:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall:** It provides the ratio of correctly identified vulnerabilities to the total number of actual vulnerabilities in the dataset. It is defined by the following equation:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1 Score:** It is a measure that combines precision and recall, providing a balanced measure of the models performance. It is defined by the following formula:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Analyzing Results

We compared the evaluation metric results of our fine-tuned model and conducted a comparative analysis of recent research in the field of vulnerability detection. Specifically, we focused on the study by Bakhshandeh et al. (2023) [10] in which the authors compared the performance of chatPT-3.5 against established vulnerability detection tools like Semgrep, Bandit, and SonarQube. Table 4 presents a comparison of our fine-tuned model: Vulscanner performance metrics against those reported by Bakhshandeh et al. for ChatGPT-3.5 and traditional SAST tools.

TABLE 4. Comparison of Evaluation Metrics

	Precision	Recall	F1
Semgrep	0.4682	0.1123	0.1812
SonarQube	0.3283	0.0419	0.0743
Bandit	0.3168	0.0609	0.1022
GPT-3.5	0.3350	0.1238	0.1808
Vulscanner	0.552	0.4487	0.6302

To provide a more granular analysis, we calculated our model F1-Score for the top 6 Common Weakness Enumerations and compared them with results reported by Bakhshandeh et al. (2023) [10] illustrated in figure 7

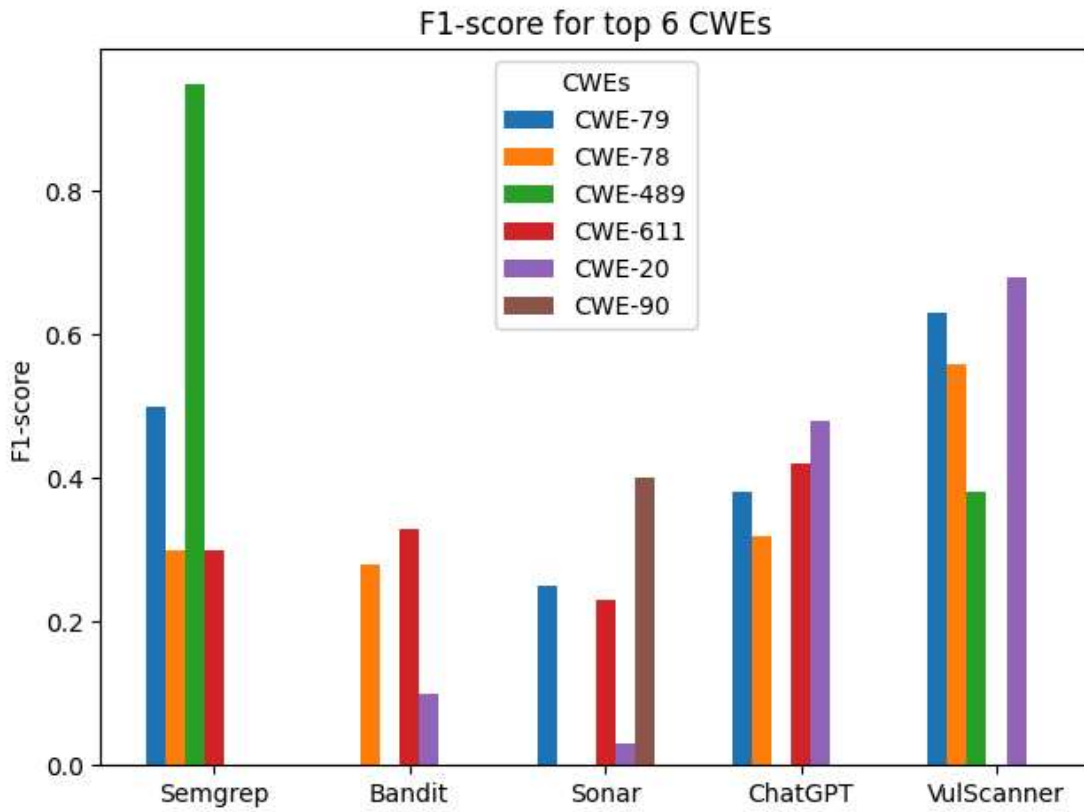


FIGURE 7. Top 6 CWEs Comparative Analysis

CHAPTER 5

VULSCANNER - SPECIALIZED VULNERABILITY DETECTION APPLICATION

On what is fear: non-acceptance of uncertainty. If we accept that uncertainty it becomes an adventure!

— Jalāl al-Dīn Muḥammad Rūmī

5.1 Introduction

We developed Vulscanner an application to detect vulnerabilities in code, utilizing our fine-tuned model. We designed the architecture of the application on the Django framework, which helped seamless integration of the model with a user-friendly frontend, providing users with specialized tools for code security analysis and correction. The application comprises of two main modules:

- Chatbot: An interactive interface where users can directly input code snippets for immediate vulnerability analysis
- File Scanning System: A robust system allowing users to upload entire Python file for comprehensive vulnerability assessment and report generation

VULSCANNER represents a significant advancement in automated code security analysis, offering real-time insights and recommendations to enhance code safety. The application features a streamlined workflow designed for optimal performance and user experience. Two main interfaces are available for users to interact with the system: the file upload system for thorough file scanning or the chatbot for real-time code analysis. Once the user input is received, it undergoes backend processing where our fine-tuned model analyzes the code for potential vulnerabilities. The model checks the code through CWE for uniformity in the classification of vulnerabilities and standards. From this analysis, therefore, the system will provide detailed feedback on the status of the code vulnerabilities, descriptions of issues found, and secure code alternatives to address the

vulnerabilities. The information is then presented to the user in a neat, organized format interactive response through the chatbot interface for code snippets or a comprehensive PDF report for the full file scan-scanning providing them with actionable insights into how to improve the security of their code.

Chatbot Module

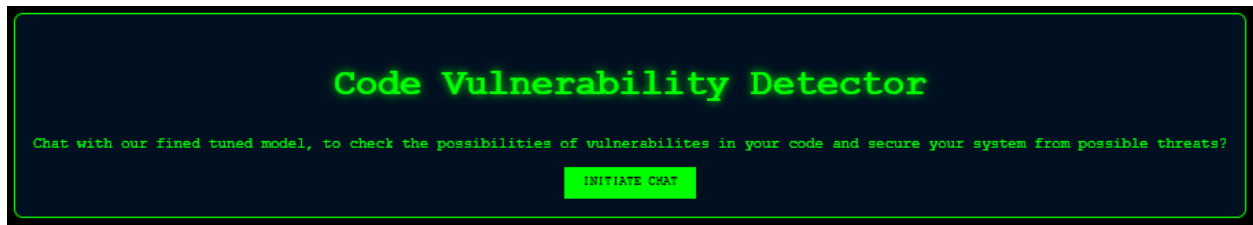
The chatbot module of our application offers a web-based interface for real-time code vulnerability analysis, providing an interactive platform where users can engage directly with our fine-tuned model, the chatbot application operates with the following workflow:

- **User Input:** Users enter their code snippet into the chat interface.
- **Backend Processing:** The code is sent to the backend of our model to be analyzed for potential vulnerabilities.
- **Response Formatting:** Custom prompts have been designed to structure the output in a readable format in the following structure if the code is vulnerable:
 - CWE-ID of the identified vulnerability.
 - Reason and description of the related cause of vulnerability.
 - Secure Version of the code

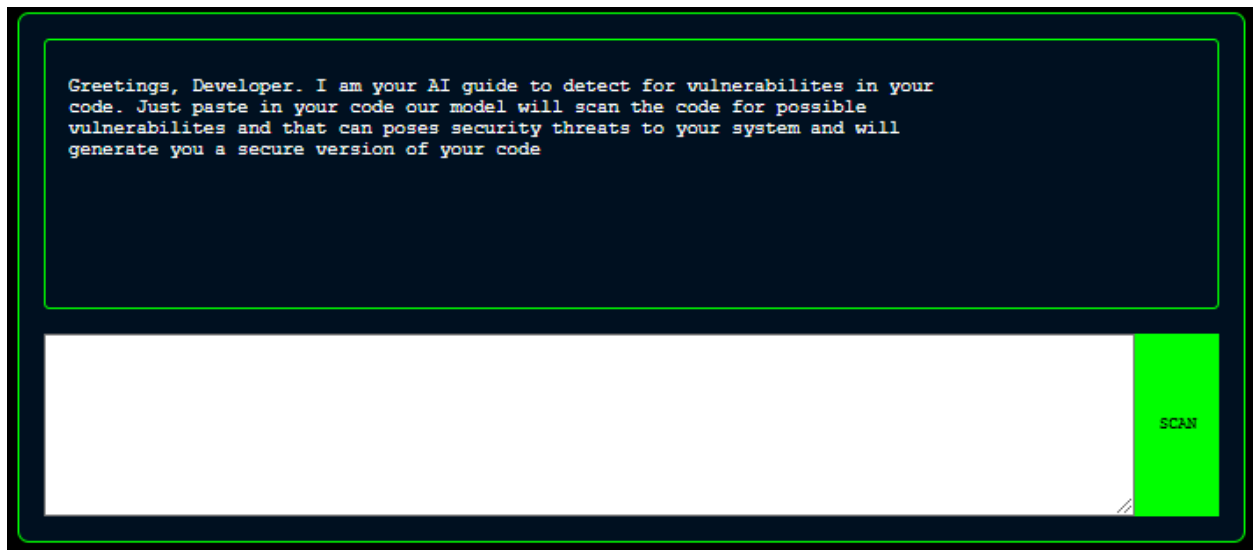
While if the provided code is secure, a simple description stating that the provided code is secure is displayed.

The user interface of our chatbot module is shown in Figure 8, which highlights its simple and user-friendly layout. The users can easily input code snippets to get responses about possible vulnerabilities in real time. The chat-like structure offers a comfortable and engaging user experience, enhancing the interaction and accessibility of the code analysis process. Figure 10 in

the appendix provides a comprehensive overview of the functionality of the chatbot, including sample interactions and outputs of how the chatbot would work its way, step by step, through the analysis of a given code snippet, detecting vulnerabilities, and then providing structured feedback including CWE-IDs, descriptions of vulnerabilities, and secure code alternatives where applicable. This graphical representation highlights very succinctly how this proposed chatbot will be capable of providing instant, actionable insights on improving code security.



(a)



(b)

FIGURE 8. Chatbot User Interface

File Scanner Module

This file scanner module of our application allows users to upload entire Python files for comprehensive vulnerability analysis reports. Provides a user-friendly interface where user can upload their files, and in the backend, our system analyzes the file and processes it to feed it to our

fine-tuned model, the file scanner module operates with the following workflow:

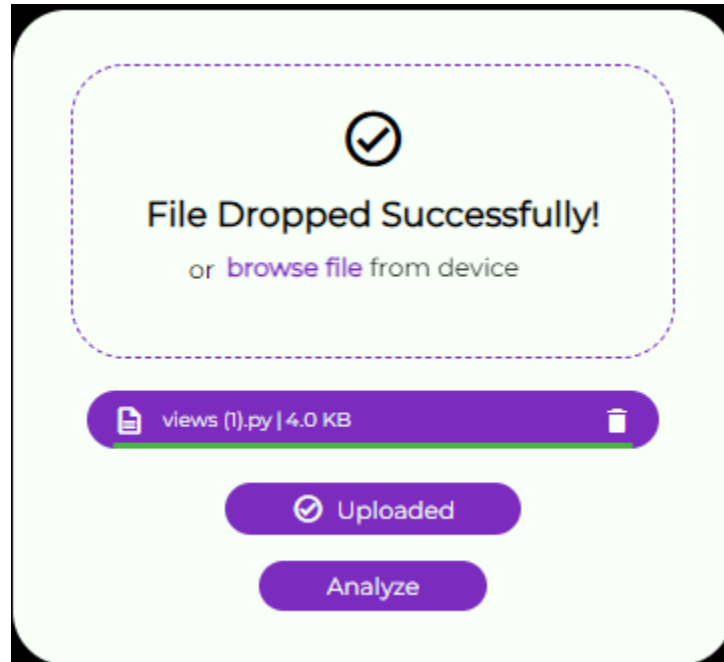


FIGURE 9. File Scanner User Interface

- File Upload: Users upload a Python file through the web interface.
- File Processing: The backend processes the file, breaking it down into chunks of individual functions and classes.
- Data Structuring: A data frame is created, storing function names and their corresponding code in a synchronous order.
- Iterative Analysis: Each function is analyzed separately for possible vulnerabilities.
- JSON Response: The model is prompted to provide analysis results in a structured JSON format.
- Dataframe Update: Results are extracted from the JSON and used to update the dataframe.
- Report Generation: A comprehensive PDF report is generated summarizing the analysis.

Figure 9 presents the file scanner system's user interface, showcasing the user-friendly layout that makes it simple for users to upload their Python files for analysis. The interface is kept simple and efficient to allow the user to start the vulnerability scanning process quickly and efficiently. For a detailed view of the output generated by our file scanner module, refer to Figure 11 in Appendix. This figure is a sample PDF report of a Python file that was generated as output after processing a source Python file. As can be seen, our vulnerability analysis is quite comprehensive in that it is able to show the full dissection of each function in the file, with highlighted vulnerabilities, their corresponding CWE-IDs, and suggested secure alternatives, thereby presenting actionable information to users for improving their code security.

CHAPTER 6

CONCLUSIONS AND FUTURE WORKS

Machine learning and AI are not a panacea for security, but they are powerful tools that can enhance our ability to detect and respond to threats.

— Nicole Eagan, CEO of Darktrace

6.1 Conclusion

In this thesis, novel methods were designed to fix the evident gaps in detecting and correcting vulnerabilities in code. This research demonstrates how fine-tuning large language models can enhance code security by utilizing custom datasets, providing custom prompts, and utilizing advanced frameworks. This study addresses the need for a specific vulnerability dataset, it also addresses critical challenges such as high false positive and negative rates, and difficulty pinpointing specific vulnerabilities and the cause of the vulnerability targeting vulnerable statements. Ultimately our approach aims to mitigate risks associated with software vulnerabilities, thereby improving overall cybersecurity measures in the software development process.

Chapter 2 laid the foundation of our study providing an overview, focusing on the background of the work, and detailing the methodology. Providing deep insights for previous related work and highlighting our contributions to the existing work.

Chapter 3 is focused on the critical task of dataset exploration and enhancement. This study specifically tailored efforts to vulnerabilities in robotic systems, a growing concern in the field of cybersecurity. This chapter discusses methods for dataset extension using synthetic and non-synthetic methods. It also provides details of processing, structuring, and formatting the dataset to be compatible with large language models, setting the stage for subsequent model training.

Chapter 4 provides a detailed description of fine-tuning large language models on custom vulnerability datasets, focusing on different prompts and training the model to detect vulnerabilities

in code. It also presents a comparative analysis with other models, demonstrating the effectiveness of this approach.

Chapter 5 introduced VulscannerAi, the practical application of this research. This specialized tool, built on the fine-tuned model, offers a user-friendly interface for vulnerability detection. The focus was on two main modules, a vulnerability chatbot for real-time code analysis and a file scanning system for comprehensive vulnerability assessment.

6.2 Research Questions

Q1: How does the fine-tuned model’s performance vary across different programming languages commonly used in robotic systems?

Further testing and fine-tuning would be required to evaluate performance across several languages typically used in robotic systems, such as C++ or code particular to ROS. It may be necessary to gather new datasets and go through training procedures for each target language, but this offers a chance for future research to broaden the model’s coverage of programming languages significant to robotics.

Q2: What is the impact of dataset size and diversity on the model’s performance, and how can we optimize data collection strategies for future improvements?

Generally, larger and more diverse datasets are better. Optimization can also be performed in the way data is collected like gathering a wider range of types of vulnerabilities, balancing different CWEs in the dataset, and adding real-world code samples derived from various robotic systems.

Q3: How does the model’s performance compare to human experts in identifying novel or complex vulnerabilities in robotic system code?

While this model may perform better in the case of known patterns of vulnerabilities, humans might turn out to be more effective in detecting novel and complex vulnerabilities. A comparison study will thus be essential for quantifying such differences.

Q4: Can the model’s explanations for identified vulnerabilities be improved to provide more actionable insights for developers?

Certainly, by adding more context-specific details, delivering code-level annotations, and providing detailed corrective recommendations based on robotic system security best practices, the model's explanations might be improved.

Future research directions aimed at expanding the model's applicability and improving its skills in automated vulnerability identification will be guided by these questions.

6.3 Future Works

Building upon the foundation of the work presented in this study, future research endeavors should focus on the enhancement on technical aspects for vulnerability detection such as:

- **IDE Integration:** Developing plugins for code editors like Visual Studio Code to enable real-time vulnerability scanning as the developers write the code. This integration can provide immediate and parallel feedback, allowing users to correct the code while writing.
- **Expanded Language Support:** Extending the model's ability, to support multiple languages other than Python, making it a more versatile tool for diverse development environments.
- **Continuous Learning Model:** Implementing model for continuous model improvement, by targeting transfer learning from the produced results.
- **Performance Optimization:** Improving the model and application to handle larger codebases more quickly, which will make it more appropriate for enterprise-level applications.

APPENDIX

```
Greetings, Developer. I am your AI guide to detect for vulnerabilities in your code. Just paste in your code our model will scan the code for possible vulnerabilities and that can poses security threats to your system and will generate you a secure version of your code

'''Develop a function to log sensor readings from a robotic system, ensuring it handles variable-length input safely.'''
def log_sensor_readings(readings):
    buffer = [0] * 6
    for i in range(len(readings)):
        buffer[i] = readings[i] # Potential overflow
    logging_system.log(buffer)

The above code is **Insecure**.

**CVE-Id:** CVE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer)

**Reason:** The code does not check if the input 'readings' is within the bounds of the 'buffer' array. If the length of 'readings' is greater than 6, it will cause an overflow, potentially leading to a buffer overflow vulnerability.

**Secure Version:**

'''python
def log_sensor_readings(readings):
    """
    Logs sensor readings from a robotic system.

    Args:
        readings (list): A list of sensor readings.

    Raises:
        ValueError: If the input list is too long.
    """
    if len(readings) > 6:
        raise ValueError("Too many readings. Maximum allowed is 6.")
    logging_system.log(readings)

In the secure version, we first check if the length of 'readings' is greater than 6. If it is, we raise a 'ValueError' with a descriptive message. This prevents the potential buffer overflow vulnerability.

Alternatively, if you want to handle variable-length input, you can use a dynamic data structure like a list or a bytearray:

'''python
def log_sensor_readings(readings):
    """
    Logs sensor readings from a robotic system.

    Args:
        readings (list): A list of sensor readings.
    """
    logging_system.log(readings)

In this version, we simply pass the input list to the 'logging_system.log' function without any bounds checking. This assumes that the 'logging_system.log' function is designed to handle variable-length input safely.
```

FIGURE 10. Workflow of Chat Module

GENERATED REPORT

TOTAL FUNCTIONS IN FILE 8

No.	Function CODE	Secure Code	Vulnerability Status	CWE IDS
1	<pre>def task_list(request): tasks = Task.objects.all() print(tasks) return render(request, 'task_list.html', {'tasks': tasks})</pre>	<pre>{'tasks': Task.objects.all().values_list()}</pre>	The code is vulnerable to a Cross-Site Scripting (XSS) attack because it directly prints the output of the database query without any sanitization or validation. An attacker could inject malicious HTML or JavaScript code into the database, which would be executed by the browser when the page is rendered.	CWE-434: Unrestricted Include in Library File
2	<pre>def task_details(request,t_id): task = Task.objects.get(t_id=t_id) return render(request, 'task_details.html', {'task': task})</pre>	<pre>{'task': Task.objects.get(t_id=t_id.id)}; # Use t_id.id to prevent SQL injection</pre>	The code is vulnerable to SQL injection because it directly uses user input (t_id) to query the database without proper sanitization or parameterization.	CWE-89: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
3	<pre>def task_create(request): if request.method == 'POST': try: title = request.POST.get('title') description = request.POST.get('description') status = request.POST.get('status') priority = request.POST.get('priority') due_date = request.POST.get('due_date') # category = request.POST.get('category') category = TaskC ategory.objects.get(name=request.POST.get('ca tegory')) # assigned_to = request.POST.get('assigned_to') assigned_to = Member.objects.get(name=request.POST.get(' assigned_to')) print("TASK CATEGORY:---",category) task = Task.objects. create(title=title,description=description,status= status,priority=priority,due_date=due_date,cate gory=category,assigned_to=assigned_to) task.save() return redirect('task_list') except Exception as e: print(e) member = Member.objects.all() task_category = TaskCategory.objects.all() print("MEMBER---",member) return render(request, 'task_create.html',{'member':m ember,'task_category':task_category})</pre>	<pre>def task_create(request): if request.method == 'POST': try: title = request.POST.get('title', "") description = request.POST.get('description', "") status = request.POST.get('status', "") priority = request.POST.get('priority', "") due_date = request.POST.get('due_date', "") category = request.POST.get('category', "") assigned_to = request.POST.get('assigned_to', "") # Validate user input if not all([title, description, status, priority, due_date, category, assigned_to]): return render(request, 'task_create.html', {'error': 'Please fill in all fields.}) category = TaskCategory.objects.filter(name=category).first() if not category: return render(request, 'task_create.html', {'error': 'Invalid category.}) assigned_to = Member.objects.filter(name=assigned_to).first() if not assigned_to: return render(request, 'task_create.html', {'error': 'Invalid assigned to.}) task = Task.objects.create(title=title, description=description, status=status, priority=priority, due_date=due_date, category=category, assigned_to=assigned_to) task.save() return redirect('task_list') except Exception as e: print(e) member = Member.objects.all() task_category = TaskCategory.objects.all() return render(request, 'task_create.html', {'member': member, 'task_category': task_category})</pre>	The code does not validate user input properly, which can lead to potential security vulnerabilities such as SQL injection and cross-site scripting (XSS).	CWE-20: Improper Input Validation

4	<pre> def task_update(request,t_id): try: task = Task.objects.get(t_id=t_id) members = Member.objects.all() category = TaskCategoy.objects.all() print("TASK",task,t_id) if request.method == 'POST': task.title = request.POST.get('title') task.description = request.POST.get('description') task.status = request.POST.get('status') task.priority = request.POST.get('priority') task.due_date = request.POST.get('due_date') task.category = TaskCategoy.objects.get(name=request.POST. get('category')) task.assigned_to = Member.obj ects.get(name=request.POST.get('assigned_to')) task.save() print("SUCESSS_____") return redirect('task_list') return render(request, 'task_ update.html',{'task':task,'member':members,'cat egory':category}) except Exception as e: print('EXCEPTION+=====',e) </pre>	<pre> def task_update(request, t_id): try: task = Task.objects.get(t_id=t_id) members = Member.objects.all() category = TaskCategoy.objects.all() print("TASK", task,t_id) if request.method == 'POST': # Validate input here task.title = request.POST.get('title', "") task.description = request.POST.get('description', "") task.status = request.POST.get('status', "") task.priority = request.POST.get('priority', "") task.due_date = request.POST.get('due_date', "") task.category = TaskCategoy.objects.get(name=request.POST.get('category', "")) task.assigned_to = Member.objects.get(name=request.POST.get('assigned_to', "")) task.save() print("SUCESSS_____") return redirect('task_list') return render(request, 'task_update.html', {'task': task,'member': members, 'category': category}) except Exception as e: print('EXCEPTION+=====', e) </pre>	The code does not validate the input from the request, which can lead to potential security vulnerabilities such as SQL injection or cross-site scripting (XSS).	CWE-20: Improper Input Validation
5	<pre> def task_delete(request,t_id): task = Task.objects.get(t_id=t_id) task.delete() return redirect('task_list') </pre>	<pre> def task_delete(request, t_id): task = Task.objects.get(t_id=t_id) task.delete() return redirect('task_list') # Secure version from django.core.exceptions import ObjectDoesNotExist def task_delete(request, t_id): try: task = Task.objects.get(t_id=t_id) task.delete() except ObjectDoesNotExist: # Handle the case when the task does not exist pass return redirect('task_list') </pre>	The task_delete function does not validate the input t_id, which can lead to a potential SQL injection attack. An attacker can manipulate the t_id parameter to delete arbitrary tasks.	CWE-89: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
6	<pre> def users_create(request): try: if request.method == 'POST': print("USER") name = request.POST.get('name') email = request.POST.get('email') password = request.POST.get('password') user = User.obje cts.create_user(first_name=name,username=e mail,password=password) Member.objects.create(name=name, email=email,password=password,) print("USER CREATED") return redirect('user_list') return render(request, 'user_create.html') except Exception as e: print('EXCEPTION+=====',e) </pre>	<pre> {"try": "if request.method == 'POST':", "name": request.POST.get('name'), "email": request.POST.get('email'), "password": request.POST.get('password'), "user": User.objects.creat e_user(first_name=name,username=email,password=password), "Member": Member.objects.create(name=name, email=email,password=password,)); return redirect('user_list') </pre>	The code does not validate or sanitize user input, making it vulnerable to XSS attacks. Additionally, storing passwords in plain text is insecure.	CWE-89: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
7	<pre> def user_list(request): members = Member.objects.all() return render(request, 'member_list.html',{'members':members}) </pre>	<pre> def user_list(request): members = Member.objects.filter(name=request.user.name) return render(request,'member_list.html',{'members':members}) </pre>	The code does not validate or sanitize user input, making it vulnerable to cross-site scripting (XSS) attacks.	CWE-89: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

FIGURE 11. Generated PDF Report For a Python File

REFERENCES

- [1] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, A. Cheshkov, and P. Zadorozhny, “Finetuning large language models for vulnerability detection,” 2024.
- [2] Y. Ding, S. Suneja, Y. Zheng, J. Laredo, A. Morari, G. Kaiser, and B. Ray, “Velvet: a novel ensemble learning approach to automatically locate vulnerable statements,” 2022.
- [3] Wikipedia. <https://en.wikipedia.org/wiki/GitHub>, 2024. Accessed: 2024-08-27.
- [4] cvedetails. <https://www.cvedetails.com/browse-by-date.php>, 2024. Accessed: 2024-08-27.
- [5] V. Kumar, M. Anjum, V. Agarwal, and P. K. Kapur, *A Hybrid Approach for Evaluation and Prioritization of Software Vulnerabilities*, pp. 39–51. Cham: Springer International Publishing, 2023.
- [6] Y. Zhou and A. Sharma, “Automated identification of security issues from commit messages and bug reports,” in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017, (New York, NY, USA)*, p. 914–919, Association for Computing Machinery, 2017.
- [7] H. Perl, S. Dechand, M. Smith, D. Arp, F. Yamaguchi, K. Rieck, S. Fahl, and Y. Acar, “Vccfinder: Finding potential vulnerabilities in open-source projects to assist code audits,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015* (I. Ray, N. Li, and C. Kruegel, eds.), pp. 426–437, ACM, 2015.
- [8] G. Jabeen, S. Rahim, W. Afzal, D. Khan, A. A. Khan, Z. Hussain, and T. Bibi, “Machine learning techniques for software vulnerability prediction: a comparative study,” *Applied Intelligence*, vol. 52, p. 17614–17635, dec 2022.

- [9] H. Hanif and S. Maffei, “Vulberta: Simplified source code pre-training for vulnerability detection,” in *2022 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2022.
- [10] A. Bakhshandeh, A. Keramatfar, A. Norouzi, and M. M. Chekidehkhoun, “Using chatgpt as a static application security testing tool,” *The ISC International Journal of Information Security*, vol. 15, no. 3, pp. 51–58, 2023.
- [11] L. Hüther, K. Sohr, B. J. Berger, H. Rothe, and S. Edelkamp, “Machine learning for security: A lightweight and adaptable approach,” in *Computer Security – ESORICS 2023: 28th European Symposium on Research in Computer Security, The Hague, The Netherlands, September 25–29, 2023, Proceedings, Part IV*, (Berlin, Heidelberg), p. 85–104, Springer-Verlag, 2024.
- [12] A. Keramatfar, M. Rafiee, and H. Amirkhani, “Graph neural networks: A bibliometrics overview,” *Machine Learning with Applications*, vol. 10, p. 100401, 2022.
- [13] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet?,” *IEEE Transactions on Software Engineering*, vol. 48, pp. 3280–3296, sep 2022.
- [14] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, “Vulrepair: a t5-based automated software vulnerability repair,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*, (New York, NY, USA), p. 935–947, Association for Computing Machinery, 2022.
- [15] Wikipedia. https://en.wikipedia.org/wiki/WannaCry_ransomware_attack, 2024. Accessed: 2024-08-27.

- [16] Wikipedia. https://en.wikipedia.org/wiki/2017_Equifax_data_breach, 2024. Accessed: 2024-08-27.
- [17] Wikipedia. https://en.wikipedia.org/wiki/2024_CrowdStrike_incident, 2024. Accessed: 2024-08-27.
- [18] Edwar Target. <https://www.thestack.technology/analysis-of-cves-in-2022-software-vulnerabilities-cwes-most-dangerous/>, 2024. Accessed: 2024-08-27.
- [19] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” 2019.
- [20] X. Zhou, S. Cao, X. Sun, and D. Lo, “Large language model for vulnerability detection and repair: Literature review and the road ahead,” 2024.
- [21] F. Lomio, E. Iannone, A. Lucia, F. Palomba, and V. Lenarduzzi, “Just-in-time software vulnerability detection: Are we there yet?,” *Journal of Systems and Software*, vol. 188, 02 2022.
- [22] S. Liu, W. Ma, J. Wang, X. Xie, R. Feng, and Y. Liu, “Enhancing code vulnerability detection via vulnerability-preserving data augmentation,” in *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, LCTES 2024, (New York, NY, USA), p. 166–177, Association for Computing Machinery, 2024.
- [23] T. Ridnik, D. Kredo, and I. Friedman, “Code generation with alphacodium: From prompt engineering to flow engineering,” 2024.
- [24] H. Shahriar and M. Zulkernine, “Mitigating program security vulnerabilities: Approaches and challenges,” *ACM Comput. Surv.*, vol. 44, jun 2012.

- [25] S. M. Ghaffarian and H. R. Shahriari, “Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey,” *ACM Comput. Surv.*, vol. 50, aug 2017.
- [26] L. Wartschinski, Y. Noller, T. Vogel, T. Kehrer, and L. Grunske, “Vudenc: Vulnerability detection with deep learning on a natural codebase for python,” *Information and Software Technology*, vol. 144, p. 106809, 2022.
- [27] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, “ μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection,” *IEEE Transactions on Dependable and Secure Computing*, vol. 18, pp. 2224–2236, sep 2021.
- [28] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, “An empirical study of deep learning models for vulnerability detection,” in *Proceedings of the 45th International Conference on Software Engineering, ICSE ’23*, p. 2237–2248, IEEE Press, 2023.
- [29] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, “Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection,” 2023.
- [30] A. Cheshkov, P. Zadorozhny, and R. Levichev, “Evaluation of chatgpt model for vulnerability detection,” 2023.
- [31] Y. Li, S. Wang, and T. N. Nguyen, “Vulnerability detection with fine-grained interpretations,” in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE ’21*, ACM, Aug. 2021.
- [32] L. Allodi and F. Massacci, “Security events and vulnerability data for cybersecurity risk estimation: Cybersecurity risk estimation,” *Risk Analysis*, vol. 37, pp. 1606–1627, 08 2017.

- [33] R. Kaur, D. Gabrijelčič, and T. Klobučar, “Artificial intelligence for cybersecurity: Literature review and future research directions,” *Information Fusion*, vol. 97, p. 101804, 2023.
- [34] Cloudflare. <https://www.cloudflare.com/en-gb/the-net/ai-vulnerabilities/>, 2024. Accessed: 2024-08-27.